Lecture 20: More Approximation Algorithms

Topics: k-Clustering · Metric TSP · MAX-CUT

Review: Approximation Algorithm Framework

Recall that an α-approximation algorithm for a minimization problem satisfies

$$\frac{\mathrm{ALG}(x)}{\mathrm{OPT}(x)} \leq \alpha \qquad \text{for some } \alpha \geq 1.$$

Template for design and analysis:

1. Prove a *lower bound* on OPT (for minimization). Hopefully the bound is easy to compute.
2. Find an efficient algorithm whose value you can relate to this bound.

Example (Vertex Cover): $\mathrm{OPT} \geq |M|$ for any matching M. A kind of greedy algorithm achieves value $2|M|$ for some matching M, giving $\mathrm{ALG}/\mathrm{OPT} \leq 2$.

k-Clustering

Problem. Given data points, separate them into k groups so that points in the same group are "similar." This is a basic subroutine in data science. "Similar" depends on the application, but it is convenient to make this numerical via a metric.

Definition 1.1 (Metric)**.** Given a set of points X, a function $d : X \times X \to \mathbb{R}_{\geq 0}$ is a **metric** if:

1. **Symmetry:** $d(x, y) = d(y, x)$,
2. **Identity of indiscernibles:** $d(x, y) = 0$ iff $x = y$,
3. **Triangle inequality:** $d(x, y) \leq d(x, w) + d(w, y)$.

Examples (for $X = \mathbb{R}^n$):

1. Euclidean: $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
2. L^1 metric: $d(x, y) = \sum_{i=1}^n |x_i - y_i|$
3. L^∞ metric: $d(x, y) = \max_i |x_i - y_i|$

If $X = \{x_1, \ldots, x_n\}$ is a finite set, a metric can be specified by a matrix of $\binom{n}{2}$ numbers.

Definition 1.2 (k-Clustering Problem)**.** Given $X = \{x_1, \ldots, x_n\}$ with a metric $d(\cdot, \cdot)$, find a partition of X into k sets $S_1, S_2, \ldots, S_k$ to **minimize the maximum diameter**:

$$\min \max_j \max_{x,y \in S_j} d(x, y).$$

k-Clustering is NP-hard.

Lower Bound on OPT

The bad case is spread-apart points. Suppose we have $k+1$ points $y_1, y_2, \ldots, y_{k+1} \in X$ such that $d(y_i, y_j) \geq r$ for all $i \neq j$ (a set of $k+1$ points with pairwise distances at least r). In any partition $S_1, \ldots, S_k$ of X, by the **Pigeonhole Principle** some set S_i must contain at least two of the y_j's. That set has diameter $\geq r$. Therefore:

$$\mathrm{OPT} \geq r.$$

The Greedy Farthest-First Algorithm

Idea. Find k centers (points in X) and assign each $x \in X$ to its nearest center.

We want to pick centers that are far from each other.

Algorithm 1 Greedy Farthest-First k-Clustering

```
Pick c_1 ∈ X arbitrarily. Let C = {c_1}.
while |C| < k do
    Let x* = arg max_{x∈X} min_{c∈C} d(x, c)    (the point farthest from C)
    C ← C ∪ {x*}
end while
Assign each x ∈ X to its closest center in C.
```

Theorem 1.1. *The Greedy Farthest-First algorithm is a* ***2-approximation*** *for k-Clustering.*

Proof. Let $C = \{c_1, \ldots, c_k\}$ be the k centers chosen. Imagine adding one more center: let c_{k+1} be the point at farthest distance from C, and let $r = d(c_{k+1}, C)$.

Claim 1: $\forall\, x \in X,\ d(x, C) \leq r$.

This follows since r is the maximum distance (the algorithm already chose the farthest point at each step).

Now consider any cluster S_j produced by the algorithm, and two points $x, y \in S_j$. Both are assigned to the same center $c_j \in C$, so $d(x, c_j) \leq r$ and $d(y, c_j) \leq r$. By the triangle inequality:

$$d(x, y) \leq d(x, c_j) + d(c_j, y) \leq 2r.$$

Therefore $\mathrm{diameter}(S_j) \leq 2r$ for all j, so $\mathrm{ALG} \leq 2r$.

Claim 2: Let $C' = \{c_1, c_2, \ldots, c_{k+1}\}$. Then $d(c_i, c_j) \geq r$ for all $i \neq j$.

This holds because we always picked the next center to maximize distance from current centers. So C' is a set of $k+1$ points with pairwise distances $\geq r$. By the lower bound argument above:

$$\mathrm{OPT} \geq r.$$

Combining: $\mathrm{ALG} \leq 2r \leq 2 \cdot \mathrm{OPT}$, so $\dfrac{\mathrm{ALG}}{\mathrm{OPT}} \leq 2$. $\square$ $\square$

Metric Travelling Salesman Problem (TSP)

Input. A set $X = \{x_1, \ldots, x_n\}$ of n points with a metric $d(\cdot, \cdot)$.

Goal. Find a cycle visiting each point exactly once to minimize the sum of distances on edges.

Metric TSP is NP-hard. We give a **2-approximation**.

Lower Bound on OPT

Take any optimal TSP solution C^* and remove one edge; we get a spanning tree T. Therefore:

$$\mathrm{OPT} = w(C^*) \geq w(T) \geq \mathrm{MST},$$

where MST denotes the weight of the minimum spanning tree.

The MST-based Algorithm

Algorithm 2 2-Approximation for Metric TSP

1: Compute a minimum spanning tree T.
2: Trace around the edges of T (DFS traversal) to produce an Eulerian circuit C.
(C visits each point at least once and uses every edge exactly twice, so $w(C) = 2\,w(T) \leq 2\,\mathrm{MST}$.)
3: **Shortcut:** walk through C in order; whenever a vertex would be revisited, skip it.
4: Output the resulting cycle C' that visits every vertex exactly once.

Analysis. By the triangle inequality, shortcuts never increase the total weight:

$$w(C') \leq w(C) = 2\,w(T) \leq 2\,\mathrm{MST} \leq 2\,\mathrm{OPT}.$$

Therefore $\dfrac{\mathrm{ALG}}{\mathrm{OPT}} \leq 2$. □

MAX-CUT

Problem. Given an undirected graph $G = (V, E)$ with non-negative edge weights $w_e \geq 0$, find a cut (A, B) with $A \cup B = V$, $A \cap B = \emptyset$ to **maximize**:

$$\sum_{e \in E(A,B)} w_e,$$

where $E(A, B)$ is the set of edges crossing the cut.

Upper Bound on OPT

The most trivial upper bound: every edge either crosses the cut or it doesn't, so

$$\mathrm{OPT} \leq \sum_{e \in E} w_e.$$

Integer Programming Formulation

Assign each vertex $v \in V$ a variable $x_v \in \{-1, +1\}$, where $+1$ means $v \in A$ and -1 means $v \in B$. An edge $e = (u, v)$ crosses the cut iff $x_u \neq x_v$, which happens iff $x_u x_v = -1$. In this case the contribution to the cut is $w_{uv} \cdot \frac{1 - x_u x_v}{2} = w_{uv}$ (since $1 - (-1) = 2$, divided by 2 gives 1); if $x_u x_v = 1$ the contribution is 0. So MAX-CUT can be written as:

$$\text{maximize} \quad \sum_{e=(u,v)\in E} \frac{1 - x_u x_v}{2} w_{uv} \qquad \text{subject to } x_v \in \{-1, +1\} \; \forall\, v \in V.$$

Randomized $\frac{1}{2}$-Approximation

Algorithm. For each vertex v, independently set $x_v = +1$ with probability $\frac{1}{2}$ and $x_v = -1$ with probability $\frac{1}{2}$.

Analysis. Since expectation is linear:

$$\mathbb{E}\left[\sum_{e=(u,v)} \frac{1 - x_u x_v}{2} w_{uv}\right] = \sum_{e=(u,v)} w_{uv} \cdot \frac{1 - \mathbb{E}[x_u x_v]}{2}.$$

Since x_u and x_v are chosen independently and each is ± 1 with equal probability, $\mathbb{E}[x_u x_v] = \mathbb{E}[x_u]\,\mathbb{E}[x_v] = 0 \cdot 0 = 0$. Therefore:

$$\mathbb{E}[\text{ALG}] = \frac{1}{2} \sum_{e \in E} w_e \geq \frac{1}{2}\,\text{OPT}.$$

So in expectation this algorithm is a $\frac{1}{2}$-approximation (for a maximization problem).

Derandomization via the Method of Conditional Expectations

We want to turn the randomized algorithm into a *deterministic* $\frac{1}{2}$-approximation. The technique is the **method of conditional expectations**: we order the vertices $v_1, v_2, \ldots, v_n$ and fix $x_{v_i} \in \{-1, +1\}$ one at a time, always choosing the value that keeps the conditional expected cut weight at least $\frac{1}{2} \sum_e w_e$.

Why is this possible? At step i, the conditional expected cut weight is the average of the conditional expectations under $x_{v_i} = +1$ and $x_{v_i} = -1$. At least one of these two values must be $\geq$ the average, so we can always greedily pick the better choice.

Computing the conditional expectation at step i. Once $x_{v_1}, \ldots, x_{v_{i-1}}$ have been fixed, the conditional expected cut weight is:

$$\underbrace{\sum_{\substack{e=(v_k,v_\ell)\in E \\ k,\ell<i}} \frac{1 - x_{v_k} x_{v_\ell}}{2} w_{v_k v_\ell}}_{\text{fully determined edges}} + \underbrace{\frac{1}{2} \sum_{\substack{e\in E \\ \text{other edges}}} w_e}_{\text{remaining random edges}} .$$

The first sum is over edges where both endpoints are already fixed (their contribution is deterministic). The second sum is over all remaining edges, each of which independently contributes $w_e/2$ in expectation (since for any such edge at least one endpoint is still random). We compute this quantity for both choices $x_{v_i} \in \{+1, -1\}$ and pick the larger one.

Conclusion. At every step, the conditional expectation stays $\geq \frac{1}{2}\sum_e w_e$. After all n vertices are fixed, the result is deterministic and equals the final conditional expectation, which satisfies:

$$\mathrm{ALG} \geq \frac{1}{2}\sum_{e \in E} w_e \geq \frac{1}{2}\,\mathrm{OPT}.$$

This gives a **deterministic $\frac{1}{2}$-approximation** for MAX-CUT. □

Remark. There is a better approximation known for MAX-CUT: the **Goemans–Williamson algorithm** achieves a 0.878-approximation. It involves *semidefinite programming*.